

DATA RETRIEVAL LANGUAGE (DRL)

1. Syntax: `SELECT <set of column names> FROM <table_name>;`
2. Order of execution from RIGHT to LEFT.
3. Q Can we use SELECT keyword without using FROM clause?
 1. Yes, using DUAL Tables.
 2. Dual tables are dummy tables created by MySQL, help users to do certain obvious actions without referring to user defined tables.
 3. eg., `SELECT 55+11;`
`SELECT now();`
`SELECT ucase();` etc.
4. WHERE
 1. Reduce rows based on given conditions.
 2. Eg, `SELECT * FROM customer WHERE age > 18;`
5. BETWEEN
 1. `SELECT * FROM customer WHERE age between 0 AND 100;`
 2. In the above eg, 0 and 100 are inclusive.
6. IN
 1. Reduces OR conditions;
 2. eg, `SELECT * FROM officers WHERE officer_name IN ('Lakshay', 'Maharana Pratap', 'Deepika');`
7. AND/OR/NOT
 1. AND: `WHERE cond1 AND cond2`
 2. OR: `WHERE cond1 OR cond2`
 3. NOT: `WHERE col_name NOT IN (1,2,3,4);`
8. IS NULL
 1. eg, `SELECT * FROM customer WHERE prime_status is NULL;`
9. Pattern Searching / Wildcard ('%', '_')
 1. '%', any number of character from 0 to n. Similar to '*' asterisk in regex.
 2. '_', only one character.
 3. `SELECT * FROM customer WHERE name LIKE '%p_';`
10. ORDER BY
 1. Sorting the data retrieved using WHERE clause.
 2. `ORDER BY <column-name> DESC;`
 3. DESC – Descending and ASC – Ascending
 4. eg, `SELECT * FROM customer ORDER BY name DESC;`
11. GROUP BY
 1. GROUP BY Clause is used to collect data from multiple records and group the result by one or more column. It is generally used in a SELECT statement.
 2. Groups into category based on column given.
 3. `SELECT c1, c2, c3 FROM sample_table WHERE cond GROUP BY c1, c2, c3.`
 4. All the column names mentioned after SELECT statement shall be repeated in GROUP BY, in order to successfully execute the query.
 5. Used with aggregation functions to perform various actions.
 1. COUNT()
 2. SUM()
 3. AVG()
 4. MIN()
 5. MAX()
12. DISTINCT
 1. Find distinct values in the table.
 2. `SELECT DISTINCT(col_name) FROM table_name;`
 3. GROUP BY can also be used for the same
 1. "Select col_name from table GROUP BY col_name;" same output as above DISTINCT query.

LEC-9: SQL in 1-Video



1. SQL: Structured Query Language, used to access and manipulate data.
2. SQL used **CRUD** operations to communicate with DB.
 1. **CREATE** - execute INSERT statements to insert new tuple into the relation.
 2. **READ** - Read data already in the relations.
 3. **UPDATE** - Modify already inserted data in the relation.
 4. **DELETE** - Delete specific data point/tuple/row or multiple rows.
3. SQL is **not** DB, is a **query language**.
4. What is **RDBMS**? (Relational Database Management System)
 1. Software that enable us to implement designed relational model
 2. e.g., MySQL, MSSQL, Oracle, IBM etc.
 3. Table/Relation is the simplest form of data storage object in R-DB.
 4. MySQL is open-source RDBMS, and it uses SQL for all CRUD operations
5. MySQL used client-server model, where client is CLI or frontend that used services provided by MySQL server.
6. **Difference between SQL and MySQL**
 1. SQL is Structured Query language used to perform CRUD operations in R-DB, while MySQL is a RDBMS used to store, manage and administrate DB (provided by itself) using SQL.

SQL DATA TYPES (Ref: https://www.w3schools.com/sql/sql_datatypes.asp)

1. In SQL DB, data is stored in the form of tables.
2. Data can be of different types, like INT, CHAR etc.

DATATYPE	Description
CHAR	string(0-255), string with size = (0, 255), e.g., CHAR(251)
VARCHAR	string(0-255)
TINYTEXT	String(0-255)
TEXT	string(0-65535)
BLOB	string(0-65535)
MEDIUMTEXT	string(0-16777215)
MEDIUMBLOB	string(0-16777215)
LONGTEXT	string(0-4294967295)
LOBLOB	string(0-4294967295)
TINYINT	integer(-128 to 127)
SMALLINT	integer(-32768 to 32767)
MEDIUMINT	integer(-8388608 to 8388607)
INT	integer(-2147483648 to 2147483647)
BIGINT	integer (-9223372036854775808 to 9223372036854775807)
FLOAT	Decimal with precision to 23 digits
DOUBLE	Decimal with 24 to 53 digits

DATATYPE	Description
DECIMAL	Double stored as string
DATE	YYYY-MM-DD
DATETIME	YYYY-MM-DD HH:MM:SS
TIMESTAMP	YYYYMMDDHHMMSS
TIME	HH:MM:SS
ENUM	One of the preset values
SET	One or many of the preset values
BOOLEAN	0/1
BIT	e.g., BIT(n), n upto 64, store values in bits.

3. Size: TINY < SMALL < MEDIUM < INT < BIGINT.
4. Variable length Data types e.g., VARCHAR, are better to use as they occupy space equal to the actual data size.
5. Values can also be unsigned e.g., INT UNSIGNED.
6. Types of SQL commands:
 1. DDL (data definition language): defining relation schema.
 1. CREATE: create table, DB, view.
 2. ALTER TABLE: modification in table structure. e.g. change column datatype or add/remove columns.
 3. DROP: delete table, DB, view.
 4. TRUNCATE: remove all the tuples from the table.
 5. RENAME: rename DB name, table name, column name etc.
 2. DRL/DQL (data retrieval language / data query language): retrieve data from the tables.
 1. SELECT
 3. DML (data modification language): use to perform modifications in the DB
 1. INSERT: insert data into a relation
 2. UPDATE: update relation data.
 3. DELETE: delete row(s) from the relation.
 4. DCL (Data Control language): grant or revoke authorities from user.
 1. GRANT: access privileges to the DB
 2. REVOKE: revoke user access privileges.
 5. TCL (Transaction control language): to manage transactions done in the DB
 1. START TRANSACTION: begin a transaction
 2. COMMIT: apply all the changes and end transaction
 3. ROLLBACK: discard changes and end transaction
 4. SAVEPOINT: checkout within the group of transactions in which to rollback.

MANAGING DB (DDL)

1. Creation of DB
 1. CREATE DATABASE IF NOT EXISTS db-name;
 2. USE db-name; //need to execute to choose on which DB CREATE TABLE etc commands will be executed.
//make switching between DBs possible.
 3. DROP DATABASE IF EXISTS db-name; //dropping database
 4. SHOW DATABASES; //list all the DBs in the server.
 5. SHOW TABLES; //list tables in the selected DB.

- SQL is smart enough to realise that if you are using GROUP BY and not using any aggregation function, then you mean "DISTINCT".

13. GROUP BY HAVING

- Out of the categories made by GROUP BY, we would like to know only particular thing (cond).
- Similar to WHERE.
- Select COUNT(cust_id), country from customer GROUP BY country HAVING COUNT(cust_id) > 50;
- WHERE vs HAVING
 - Both have same function of filtering the row base on certain conditions.
 - WHERE clause is used to filter the rows from the table based on specified condition.
 - HAVING clause is used to filter the rows from the groups based on the specified condition.
 - HAVING is used after GROUP BY while WHERE is used before GROUP BY clause.
 - If you are using HAVING, GROUP BY is necessary.
 - WHERE can be used with SELECT, UPDATE & DELETE keywords while GROUP BY used with SELECT.

CONSTRAINTS (DDL)

1. Primary key



- PK is not null, unique and only one per table.

```
id INT PRIMARY KEY,
delivery_date DATE,
order_placed_date DATE,
```

```
);
```

```
email VARCHAR(1024) UNIQUE,
```

```
);
```

```
CONSTRAINT age_check CHECK (age > 12),
```

```
);
```



```
FOREIGN KEY(cust_id) REFERENCES customer(id) ON DELETE CASCADE
);
```

3. ON DELETE NULL - (can FK have null values?)

```
1. CREATE TABLE ORDER (
    order_id int PRIMARY KEY,
    delivery_date DATE,
    cust_id INT,
    FOREIGN KEY(cust_id) REFERENCES customer(id) ON DELETE SET NULL
);
```

4. REPLACE

1. Primarily used for already present tuple in a table.
2. As UPDATE, using REPLACE with the help of WHERE clause in PK, then that row will be replaced.
3. As INSERT, if there is no duplicate data new tuple will be inserted.
4. REPLACE INTO student (id, class) VALUES(4, 3);
5. REPLACE INTO table SET col1 = val1, col2 = val2;

JOINING TABLES

1. All RDBMS are relational in nature, we refer to other tables to get meaningful outcomes.

2. FK are used to do reference to other table.

3. INNER JOIN

1. Returns a resultant table that has matching values from both the tables or all the tables.
2. SELECT column-list FROM table1 INNER JOIN table2 ON condition1
INNER JOIN table3 ON condition2
...;

3. Alias in MySQL (AS)

1. Aliases in MySQL is used to give a temporary name to a table or a column in a table for the purpose of a particular query. It works as a nickname for expressing the tables or column names. It makes the query short and neat.
2. SELECT col_name AS alias_name FROM table_name;
3. SELECT col_name1, col_name2, FROM table_name AS alias_name;

4. OUTER JOIN

1. LEFT JOIN

1. This returns a resulting table that all the data from left table and the matched data from the right table.
2. SELECT columns FROM table LEFT JOIN table2 ON Join_Condition;

2. RIGHT JOIN

1. This returns a resulting table that all the data from right table and the matched data from the left table.
2. SELECT columns FROM table RIGHT JOIN table2 ON join_cond;

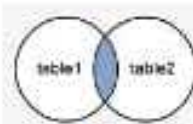
3. FULL JOIN

1. This returns a resulting table that contains all data when there is a match on left or right table data.
2. Emulated in MySQL using LEFT and RIGHT JOIN.
3. LEFT JOIN UNION RIGHT JOIN.
4. SELECT columns FROM table1 as t1 LEFT JOIN table2 as t2 ON t1.id = t2.id
UNION
SELECT columns FROM table1 as t1 RIGHT JOIN table2 as t2 ON t1.id = t2.id;
5. UNION ALL, can also be used this will duplicate values as well while UNION gives unique values.

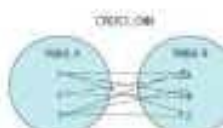
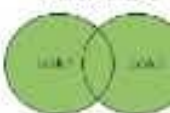
5. CROSS JOIN

1. This returns all the cartesian products of the data present in both tables. Hence, all possible variations are reflected in the output.
2. Used rarely in practical purpose.
3. Table-1 has 10 rows and table-2 has 5, then resultant would have 50 rows.
4. SELECT column-lists FROM table1 CROSS JOIN table2;

6. SELF JOIN



FULL OUTER JOIN



5. DEFAULT

1. Set default value of the column.
2. CREATE TABLE account (
...
savings-rate DOUBLE NOT NULL DEFAULT 4.25,
...
);

6. An attribute can be PK and FK both in a table.

7. ALTER OPERATIONS

1. Changes schema
2. ADD
 1. Add new column.
 2. ALTER TABLE table-name ADD new_col_name datatype ADD new_col_name_2 datatype;
 3. e.g., ALTER TABLE customer ADD age INT NOT NULL;
3. MODIFY
 1. Change datatype of an attribute.
 2. ALTER TABLE table-name MODIFY col-name col-datatype;
 3. E.g., VARCHAR TO CHAR
ALTER TABLE customer MODIFY name CHAR(1024);
4. CHANGE COLUMN
 1. Rename column name.
 2. ALTER TABLE table-name CHANGE COLUMN old-col-name new-col-name new-col-datatype;
 3. e.g., ALTER TABLE customer CHANGE COLUMN name customer-name VARCHAR(1024);
5. DROP COLUMN
 1. Drop a column completely.
 2. ALTER TABLE table-name DROP COLUMN col-name;
 3. e.g., ALTER TABLE customer DROP COLUMN middle-name;
6. RENAME
 1. Rename table name itself.
 2. ALTER TABLE table-name RENAME TO new-table-name;
 3. e.g., ALTER TABLE customer RENAME TO customer-details;

DATA MANIPULATION LANGUAGE (DML)

1. INSERT

1. INSERT INTO table-name(col1, col2, col3) VALUES (v1, v2, v3), (val1, val2, val3);

2. UPDATE

1. UPDATE table-name SET col1 = 1, col2 = 'abc' WHERE id = 1;
2. Update multiple rows e.g.,
 1. UPDATE student SET standard = standard + 1;

3. ON UPDATE CASCADE

1. Can be added to the table while creating constraints. Suppose there is a situation where we have two tables such that primary key of one table is the foreign key for another table. If we update the primary key of the first table then using the ON UPDATE CASCADE foreign key of the second table automatically get updated.

3. DELETE

1. DELETE FROM table-name WHERE id = 1;
2. DELETE FROM table-name; //all rows will be deleted.
3. DELETE CASCADE - (to overcome DELETE constraint of Referential constraints)
 1. What would happen to child entry if parent table's entry is deleted?
 2. CREATE TABLE ORDER (
order_id int PRIMARY KEY,
delivery_date DATE,
cust_id INT,



1. It is used to get the output from a particular table when the same table is joined to itself.
 2. Used very less.
 3. Emulated using INNER JOIN.
 4. `SELECT columns FROM table as t1 INNER JOIN table as t2 ON t1.id = t2.id;`
7. Join without using join keywords.
1. `SELECT * FROM table1, table2 WHERE condition;`
 2. eg, `SELECT artist_name, album_name, year_recorded FROM artist, album WHERE artistid = album.artist_id;`

SET OPERATIONS

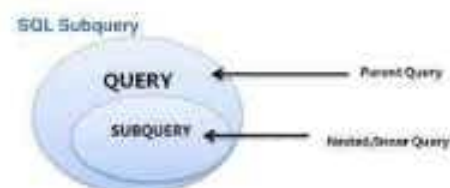
1. Used to combine multiple select statements.
2. Always gives distinct rows.

JOIN	SET Operations
Combines multiple tables based on matching condition.	Combination is resulting set from two or more SELECT statements.
Column wise combination.	Row wise combination.
Data types of two tables can be different.	Datatypes of corresponding columns from each table should be the same.
Can generate both distinct or duplicate rows.	Generate distinct rows.
The number of column(s) selected may or may not be the same from each table.	The number of column(s) selected must be the same from each table.
Combines results horizontally.	Combines results vertically.

3. UNION
1. Combines two or more SELECT statements.
 2. `SELECT * FROM table1`
`UNION`
`SELECT * FROM table2;`
 3. Number of column, order of column must be same for table1 and table2.
4. INTERSECT
1. Returns common values of the tables.
 2. Emulated.
 3. `SELECT DISTINCT column-list FROM table-1 INNER JOIN table-2 USING(join_cond);`
 4. `SELECT DISTINCT * FROM table1 INNER JOIN table2 ON USING(id);`
5. MINUS
1. This operator returns the distinct row from the first table that does not occur in the second table.
 2. Emulated.
 3. `SELECT column_list FROM table1 LEFT JOIN table2 ON condition WHERE table2.column_name IS NULL;`
 4. eg, `SELECT id FROM table-1 LEFT JOIN table-2 USING(id) WHERE table-2.id IS NULL;`

SUBQUERIES

1. Outer query depends on inner query.
2. Alternative to joins.
3. Nested queries.
4. `SELECT column_list(s) FROM table_name WHERE column_name OPERATOR (SELECT column_list(s) FROM table_name [WHERE]);`
5. eg, `SELECT * FROM table1 WHERE col1 IN (SELECT col1 FROM table1);`
6. Sub queries exist mainly in 3 clauses.
 1. Inside a WHERE clause.





2. Inside a FROM clause.
3. Inside a SELECT clause.
7. Subquery using FROM clause
 1. `SELECT MAX(rating) FROM (SELECT * FROM movie WHERE country = 'India') as temp;`
8. Subquery using SELECT
 1. `SELECT (SELECT column_list(s) FROM T_name WHERE condition), columnList(s) FROM T2_name WHERE condition;`
9. Derived Subquery
 1. `SELECT columnLists(s) FROM (SELECT columnLists(s) FROM table_name WHERE [condition]) as new_table_name;`
10. Co-related sub-queries
 1. With a normal nested subquery, the inner SELECT query runs first and executes once, returning values to be used by the main query. A correlated subquery, however, executes once for each candidate row considered by the outer query. In other words, the inner query is driven by the outer query.

```
SELECT column1, column2, ....  
FROM table1 as outer  
WHERE column1 operator  
      (SELECT column1, column2  
       FROM table2  
       WHERE expr1 =  
             outer.expr2);
```

JOIN VS SUB-QUERIES

JOINS	SUBQUERIES
Faster	Slower
Joins maximise calculation burden on DBMS	Keeps responsibility of calculation on user.
Complex, difficult to understand and implement	Comparatively easy to understand and implement.
Choosing optimal join for optimal use case is difficult	Easy.

MySQL VIEWS

1. A view is a database object that has no values. Its contents are based on the base table. It contains rows and columns similar to the real table.
2. In MySQL, the View is a virtual table created by a query by joining one or more tables. It is operated similarly to the base table but does not contain any data of its own.
3. The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.
4. `CREATE VIEW view_name AS SELECT columns FROM tables [WHERE conditions];`
5. `ALTER VIEW view_name AS SELECT columns FROM table WHERE conditions;`
6. `DROP VIEW IF EXISTS view_name;`
7. `CREATE VIEW Trainer AS SELECT ccourse_name, ctrainer, temail FROM courses c, contact t WHERE cid = t.id; (View using Join clause).`

NOTE: We can also import/export table schema from files (.csv or json).

-- Q-10. Write an SQL query to print the FIRST_NAME and LAST_NAME from Worker table into a single column COMPLETE_NAME.

-- A space char should separate them.

```
select CONCAT(first_name, ' ', last_name) AS COMPLETE_NAME from worker;
```

-- Q-11. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending.

```
select * from worker ORDER by first_name;
```

-- Q-12. Write an SQL query to print all Worker details from the Worker table order by
-- FIRST_NAME Ascending and DEPARTMENT Descending.

```
select * from worker order by first_name, department DESC;
```

-- Q-13. Write an SQL query to print details for Workers with the first name as "Vipul" and "Satish" from Worker table.

```
select * from worker where first_name IN ('Vipul', 'Satish');
```

-- Q-14. Write an SQL query to print details of workers excluding first names, "Vipul" and "Satish" from Worker table.

```
select * from worker where first_name NOT IN ('Vipul', 'Satish');
```

-- Q-15. Write an SQL query to print details of Workers with DEPARTMENT name as "Admin".

```
select * from worker where department LIKE 'Admin%';
```

-- Q-16. Write an SQL query to print details of the Workers whose FIRST_NAME contains 'a'.

```
select * from worker where first_name LIKE '%a%';
```

-- Q-17. Write an SQL query to print details of the Workers whose FIRST_NAME ends with 'a'.

```
select * from worker where first_name LIKE '%a';
```

-- Q-18. Write an SQL query to print details of the Workers whose FIRST_NAME ends with 'h' and contains six alphabets.

```
select * from worker where first_name LIKE '_____h';
```

```
CREATE DATABASE ORG;
```

```
SHOW DATABASES;
```

```
USE ORG;
```

```
CREATE TABLE Worker (
```

```
    WORKER_ID INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
```

```
    FIRST_NAME CHAR(25),
```

```
    LAST_NAME CHAR(25),
```

```
    SALARY INT(15),
```

```
    JOINING_DATE DATETIME,
```

```
    DEPARTMENT CHAR(25)
```

```
);
```

```
INSERT INTO Worker
```

```
(WORKER_ID, FIRST_NAME, LAST_NAME, SALARY, JOINING_DATE, DEPARTMENT) VALUES
```

```
(001, 'Monika', 'Arora', 100000, '14-02-20 09.00.00', 'HR'),
```

```
(002, 'Niharika', 'Verma', 80000, '14-06-11 09.00.00', 'Admin'),
```

```
(003, 'Vishal', 'Singhal', 300000, '14-02-20 09.00.00', 'HR'),
```

```
(004, 'Amitabh', 'Singh', 500000, '14-02-20 09.00.00', 'Admin'),
```

```
(005, 'Vivek', 'Bhati', 500000, '14-06-11 09.00.00', 'Admin'),
```

```
(006, 'Vipul', 'Dixan', 200000, '14-06-11 09.00.00', 'Account'),
```

```
(007, 'Satish', 'Kumar', 75000, '14-01-20 09.00.00', 'Account'),
```

```
(008, 'Geetika', 'Chauhan', 90000, '14-04-11 09.00.00', 'Admin');
```

```
SELECT * FROM Title;
```

```
CREATE TABLE Bonus (
```

```
    WORKER_REF_ID INT,
```

```
    BONUS_AMOUNT INT(10),
```

```
    BONUS_DATE DATETIME,
```

```
    FOREIGN KEY (WORKER_REF_ID)
```


-- Q-19. Write an SQL query to print details of the Workers whose SALARY lies between 100000 and 500000.

```
select * from worker where salary between 100000 AND 500000;
```

-- Q-20. Write an SQL query to print details of the Workers who have joined in Feb' 2014.

```
select * from worker where YEAR(joining_date) = 2014 AND MONTH(joining_date) = 02;
```

-- Q-21. Write an SQL query to fetch the count of employees working in the department 'Admin'.

```
select department, count(*) from worker where department = 'Admin';
```

-- Q-22. Write an SQL query to fetch worker full names with salaries >= 50000 and <= 100000.

```
select concat(first_name, ' ', last_name) from worker
```

```
where salary between 50000 and 100000;
```

-- Q-23. Write an SQL query to fetch the no. of workers for each department in the descending order.

```
select department, count(worker_id) AS no_of_worker from worker group by department  
ORDER BY no_of_worker desc;
```

-- Q-24. Write an SQL query to print details of the Workers who are also Managers.

```
select w.* from worker as w inner join title as t on w.worker_id = t.worker_ref_id where  
t.worker_title = 'Manager';
```

-- Q-25. Write an SQL query to fetch number (more than 1) of same titles in the ORG of different types.

```
select worker_title, count(*) as count from title group by worker_title having count > 1;
```

-- Q-26. Write an SQL query to show only odd rows from a table.

```
-- select * from worker where MOD (WORKER_ID, 2) != 0;
```

```
select * from worker where MOD (WORKER_ID, 2) <> 0;
```



-- Q-1. Write an SQL query to fetch "FIRST_NAME" from Worker table using the alias name as <WORKER_NAME>.

```
select first_name AS WORKER_NAME from worker;
```

-- Q-2. Write an SQL query to fetch "FIRST_NAME" from Worker table in upper case.

```
select UPPER(first_name) from worker;
```

-- Q-3. Write an SQL query to fetch unique values of DEPARTMENT from Worker table.

```
SELECT distinct department from worker;
```

-- Q-4. Write an SQL query to print the first three characters of FIRST_NAME from Worker table.

```
select substring(first_name, 1, 3) from worker;
```

-- Q-5. Write an SQL query to find the position of the alphabet ('b') in the first name column 'Amitabh' from Worker table.

```
select INSTR(first_name, 'B') from worker where first_name = 'Amitabh';
```

-- Q-6. Write an SQL query to print the FIRST_NAME from Worker table after removing white spaces from the right side.

```
select RTRIM(first_name) from worker;
```

-- Q-7. Write an SQL query to print the DEPARTMENT from Worker table after removing white spaces from the left side.

```
select LTRIM(department) from worker;
```

-- Q-8. Write an SQL query that fetches the unique values of DEPARTMENT from Worker table and prints its length.

```
select distinct department, LENGTH(department) from worker;
```

-- Q-9. Write an SQL query to print the FIRST_NAME from Worker table after replacing 'a' with 'A'.

```
select REPLACE(first_name, 'a', 'A') from worker;
```



2. Inside a FROM clause.
3. Inside a SELECT clause.
7. Subquery using FROM clause
 1. `SELECT MAX(rating) FROM (SELECT * FROM movie WHERE country = 'India') as temp;`
8. Subquery using SELECT
 1. `SELECT (SELECT column_list(s) FROM T_name WHERE condition), columnList(s) FROM T2_name WHERE condition;`
9. Derived Subquery
 1. `SELECT columnLists(s) FROM (SELECT columnLists(s) FROM table_name WHERE [condition]) as new_table_name;`
10. Co-related sub-queries
 1. With a normal nested subquery, the inner SELECT query runs first and executes once, returning values to be used by the main query. A correlated subquery, however, executes once for each candidate row considered by the outer query. In other words, the inner query is driven by the outer query.

```
SELECT column1, column2, ....
FROM table1 as outer
WHERE column1 operator
      (SELECT column1, column2
       FROM table2
       WHERE expr1 =
         outer.expr2);
```

JOIN VS SUB-QUERIES

JOINS	SUBQUERIES
Faster	Slower
Joins maximise calculation burden on DBMS	Keeps responsibility of calculation on user.
Complex, difficult to understand and implement	Comparatively easy to understand and implement.
Choosing optimal join for optimal use case is difficult	Easy.

MySQL VIEWS

1. A view is a database object that has no values. Its contents are based on the base table. It contains rows and columns similar to the real table.
2. In MySQL, the View is a virtual table created by a query by joining one or more tables. It is operated similarly to the base table but does not contain any data of its own.
3. The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.
4. `CREATE VIEW view_name AS SELECT columns FROM tables [WHERE conditions];`
5. `ALTER VIEW view_name AS SELECT columns FROM table WHERE conditions;`
6. `DROP VIEW IF EXISTS view_name;`
7. `CREATE VIEW Trainer AS SELECT c.course_name, c.trainer, t.email FROM courses c, contact t WHERE c.cid = t.id; (View using join clause).`

NOTE: We can also import/export table schema from files (.csv or json).

-- Q-19. Write an SQL query to print details of the Workers whose SALARY lies between 100000 and 500000.

```
select * from worker where salary between 100000 AND 500000;
```

-- Q-20. Write an SQL query to print details of the Workers who have joined in Feb' 2014.

```
select * from worker where YEAR(joining_date) = 2014 AND MONTH(joining_date) = 02;
```

-- Q-21. Write an SQL query to fetch the count of employees working in the department 'Admin'.

```
select department, count(*) from worker where department = 'Admin';
```

-- Q-22. Write an SQL query to fetch worker full names with salaries >= 50000 and <= 100000.

```
select concat(first_name, ' ', last_name) from worker
```

```
where salary between 50000 and 100000;
```

-- Q-23. Write an SQL query to fetch the no. of workers for each department in the descending order.

```
select department, count(worker_id) AS no_of_worker from worker group by department  
ORDER BY no_of_worker desc;
```

-- Q-24. Write an SQL query to print details of the Workers who are also Managers.

```
select w.* from worker as w inner join title as t on w.worker_id = t.worker_ref_id where  
t.worker_title = 'Manager';
```

-- Q-25. Write an SQL query to fetch number (more than 1) of same titles in the ORG of different types.

```
select worker_title, count(*) as count from title group by worker_title having count > 1;
```

-- Q-26. Write an SQL query to show only odd rows from a table.

```
-- select * from worker where MOD (WORKER_ID, 2) != 0;
```

```
select * from worker where MOD (WORKER_ID, 2) <> 0;
```

-- Q-27. Write an SQL query to show only even rows from a table.

```
select * from worker where MOD (WORKER_ID, 2) = 0;
```

-- Q-28. Write an SQL query to clone a new table from another table.

```
CREATE TABLE worker_clone LIKE worker;
```

```
INSERT INTO worker_clone select * from worker;
```

```
select * from worker_clone;
```

-- Q-29. Write an SQL query to fetch intersecting records of two tables.

```
select worker.* from worker inner join worker_clone using(worker_id);
```

-- Q-30. Write an SQL query to show records from one table that another table does not have.

-- MINUS

```
select worker.* from worker left join worker_clone using(worker_id) WHERE  
worker_clone.worker_id is NULL;
```

-- Q-31. Write an SQL query to show the current date and time.

-- DUAL

```
select curdate();
```

```
select now();
```

-- Q-32. Write an SQL query to show the top n (say 5) records of a table order by descending salary.

```
select * from worker order by salary desc LIMIT 5;
```

-- Q-33. Write an SQL query to determine the nth (say n=5) highest salary from a table.

```
select * from worker order by salary desc LIMIT 4,1;
```

-- Q-34. Write an SQL query to determine the 5th highest salary without using LIMIT keyword.

```
select salary from worker w1
```

```
WHERE 4 = (
```

```
SELECT COUNT(DISTINCT (w2.salary))
```

from worker w2

where w2.salary >= w1.salary

);

-- Q-35. Write an SQL query to fetch the list of employees with the same salary.

```
select w1.* from worker w1, worker w2 where w1.salary = w2.salary and w1.worker_id != w2.worker_id;
```

-- Q-36. Write an SQL query to show the second highest salary from a table using sub-query.

```
select max(salary) from worker  
where salary not in (select max(salary) from worker);
```

-- Q-37. Write an SQL query to show one row twice in results from a table.

```
select * from worker  
  
UNION ALL  
  
select * from worker ORDER BY worker_id;
```

-- Q-38. Write an SQL query to list worker_id who does not get bonus.

```
select worker_id from worker where worker_id not in (select worker_ref_id from bonus);
```

-- Q-39. Write an SQL query to fetch the first 50% records from a table.

```
select * from worker where worker_id <= ( select count(worker_id)/2 from worker);
```

-- Q-40. Write an SQL query to fetch the departments that have less than 4 people in it.

```
select department, count(department) as depCount from worker group by department having  
depCount < 4;
```

-- Q-41. Write an SQL query to show all departments along with the number of people in there.

```
select department, count(department) as depCount from worker group by department;
```

-- Q-42. Write an SQL query to show the last record from a table.

```
select * from worker where worker_id = (select max(worker_id) from worker);
```


-- Q-43. Write an SQL query to fetch the first row of a table.

```
select * from worker where worker_id = (select min(worker_id) from worker);
```

-- Q-44. Write an SQL query to fetch the last five records from a table.

```
(select * from worker order by worker_id desc limit 5) order by worker_id;
```

-- Q-45. Write an SQL query to print the name of employees having the highest salary in each department.

```
select w.department, w.first_name, w.salary from  
(select max(salary) as maxsal, department from worker group by department) temp  
inner join worker w on temp.department = w.department and temp.maxsal = w.salary;
```

-- Q-46. Write an SQL query to fetch three max salaries from a table using co-related subquery

```
select distinct salary from worker w1  
where 3 >= (select count(distinct salary) from worker w2 where w1.salary <= w2.salary) order by  
w1.salary desc;
```

-- DRY RUN AFTER REVISING THE CORELATED SUBQUERY CONCEPT FROM LEC-9.

```
select distinct salary from worker order by salary desc limit 3;
```

-- Q-47. Write an SQL query to fetch three min. salaries from a table using co-related subquery

```
select distinct salary from worker w1  
where 3 >= (select count(distinct salary) from worker w2 where w1.salary >= w2.salary) order by  
w1.salary desc;
```

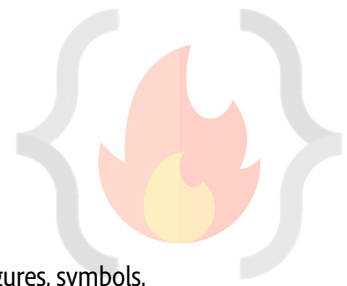
-- Q-48. Write an SQL query to fetch nth max salaries from a table.

```
select distinct salary from worker w1  
where n >= (select count(distinct salary) from worker w2 where w1.salary <= w2.salary) order by  
w1.salary desc;
```

-- Q-49. Write an SQL query to fetch departments along with the total salaries paid for each of them.

```
select department, sum(salary) as depSal from worker group by department order by depSal desc;
```

LEC-1: Introduction to DBMS



1. What is Data?

- a. Data is a collection of raw, unorganized facts and details like text, observations, figures, symbols, and descriptions of things etc.

In other words, **data does not carry any specific purpose and has no significance by itself.**

Moreover, data is measured in terms of bits and bytes – which are basic units of information in the context of computer storage and processing.

- b. Data can be recorded and doesn't have any meaning unless processed.

2. Types of Data

a. Quantitative

- i. Numerical form
- ii. Weight, volume, cost of an item.

b. Qualitative

- i. Descriptive, but not numerical.
- ii. Name, gender, hair color of a person.

3. What is Information?

- a. Info. Is **processed, organized, and structured data.**
- b. It provides **context of the data and enables decision making.**
- c. Processed data that make **sense** to us.
- d. Information is extracted from the data, by **analyzing and interpreting** pieces of data.
- e. E.g, you have data of all the people living in your locality, its Data, when you analyze and interpret the data and come to some conclusion that:
 - i. There are 100 senior citizens.
 - ii. The sex ratio is 1.1.
 - iii. Newborn babies are 100.These are information.

4. Data vs Information

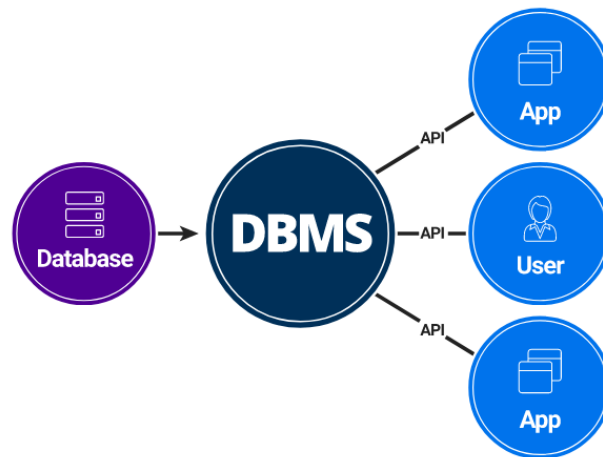
- a. Data is a collection of facts, while information puts those facts into context.
- b. While data is raw and unorganized, information is organized.
- c. Data points are individual and sometimes unrelated. Information maps out that data to provide a big-picture view of how it all fits together.
- d. Data, on its own, is meaningless. When it's analyzed and interpreted, it becomes meaningful information.
- e. Data does not depend on information; however, information depends on data.
- f. Data typically comes in the form of graphs, numbers, figures, or statistics. Information is typically presented through words, language, thoughts, and ideas.
- g. Data isn't sufficient for **decision-making**, but you can make decisions based on information.

5. What is Database?

- a. Database is an electronic place/system where data is stored in a way that it can be **easily accessed, managed, and updated.**
- b. To make real use Data, we need **Database management systems. (DBMS)**

6. What is DBMS?

- a. A database-management system (DBMS) is a collection of **interrelated data** and **a set of programs to access those data.** The collection of data, usually referred to as the **database**, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to **store and retrieve database information** that is both convenient and efficient.
- b. A DBMS is the database itself, along with all the software and functionality. It is used to perform different operations, like **addition, access, updating, and deletion** of the data.

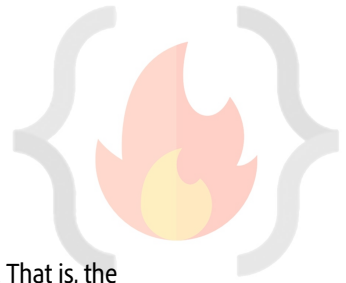


7.

8. DBMS vs File Systems

- a. **File-processing systems** has major **disadvantages**.
 - i. Data Redundancy and inconsistency
 - ii. Difficulty in accessing data
 - iii. Data isolation
 - iv. Integrity problems
 - v. Atomicity problems
 - vi. Concurrent-access anomalies
 - vii. Security problems
- b. Above 7 are also the **Advantages of DBMS** (answer to "Why to use DBMS?")

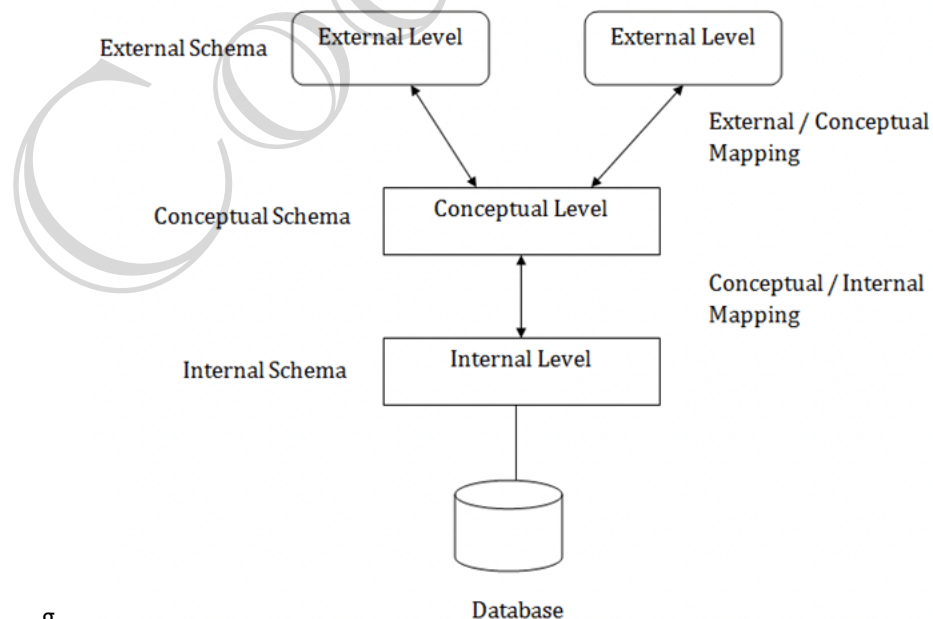
LEC-2: DBMS Architecture



1. View of Data (Three Schema Architecture)

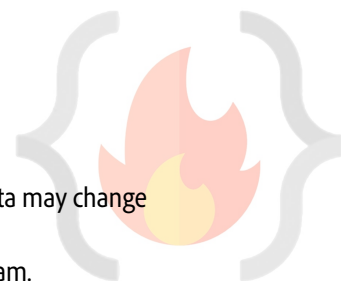
- The major purpose of DBMS is to provide users with an **abstract view** of the data. That is, the **system hides certain details of how the data is stored and maintained**.
- To simplify user interaction with the system, abstraction is applied through **several levels of abstraction**.
- The **main objective** of three level architecture is to enable multiple users to access the same data with a personalized view while storing the underlying data only once
- Physical level / Internal level**
 - The lowest level of abstraction describes how the data are stored.
 - Low-level data structures used.
 - It has **Physical schema** which describes physical storage structure of DB.
 - Talks about: Storage allocation (N-ary tree etc), Data compression & encryption etc.
 - Goal:** We must define algorithms that allow efficient access to data.
- Logical level / Conceptual level:**
 - The **conceptual schema** describes the design of a database at the conceptual level, describes **what** data are stored in DB, and what **relationships** exist among those data.
 - User at logical level does not need to be aware about physical-level structures.
 - DBA**, who must decide what information to keep in the DB use the logical level of abstraction.
 - Goal:** ease to use.
- View level / External level:**
 - Highest level of abstraction aims to simplify users' interaction with the system by providing different view to different **end-user**.
 - Each **view schema** describes the database part that a particular user group is interested and hides the remaining database from that user group.
 - At the external level, a database contains several schemas that sometimes called as **subschema**. The subschema is used to describe the different view of the database.
 - At views also provide a **security** mechanism to prevent users from accessing certain parts of DB.

three level schema



2. Instances and Schemas

- The collection of information stored in the DB at a particular moment is called an **instance** of DB.



- b. The overall design of the DB is called the DB **schema**.
 - c. Schema is **structural** description of data. Schema **doesn't change frequently**. Data may change frequently.
 - d. **DB schema** corresponds to the variable declarations (along with type) in a program.
 - e. We have 3 types of **Schemas: Physical, Logical**, several **view schemas** called subschemas.
 - f. Logical schema is most **important** in terms of its effect on application programs, as programmers construct apps by using logical schema.
 - g. **Physical data independence**, physical schema change should not affect logical schema/application programs.
3. **Data Models:**
- a. Provides a way to describe the **design** of a DB at **logical level**.
 - b. Underlying the structure of the DB is the Data Model; a collection of conceptual tools for describing **data, data relationships, data semantics & consistency constraints**.
 - c. E.g., **ER model, Relational Model, object-oriented model, object-relational data model** etc.
4. **Database Languages:**
- a. **Data definition language (DDL)** to specify the database schema.
 - b. **Data manipulation language (DML)** to express database queries and updates.
 - c. **Practically**, both language features are present in a single DB language, e.g., SQL language.
 - d. DDL
 - i. We specify consistency constraints, which must be checked, every time DB is updated.
 - e. DML
 - i. Data manipulation involves
 - 1. **Retrieval** of information stored in DB.
 - 2. **Insertion** of new information into DB.
 - 3. **Deletion** of information from the DB.
 - 4. **Updating** existing information stored in DB.
 - ii. **Query language**, a part of DML to specify statement requesting the retrieval of information.
5. **How is Database accessed from Application programs?**
- a. Apps (written in host languages, C/C++, Java) interacts with DB.
 - b. E.g., Banking system's module generating payrolls access DB by executing DML statements from the host language.
 - c. API is provided to send DML/DDL statements to DB and retrieve the results.
 - i. Open Database Connectivity (**ODBC**), Microsoft "C".
 - ii. Java Database Connectivity (**JDBC**), Java.
6. **Database Administrator (DBA)**
- a. A person who has **central control** of both the data and the programs that access those data.
 - b. **Functions** of DBA
 - i. Schema Definition
 - ii. Storage structure and access methods.
 - iii. Schema and physical organization modifications.
 - iv. Authorization control.
 - v. Routine maintenance
 - 1. Periodic backups.
 - 2. Security patches.
 - 3. Any upgrades.
7. **DBMS Application Architectures:** Client machines, on which remote DB users work, and server machines on which DB system runs.
- a. **T1 Architecture**
 - i. The client, server & DB all present on the same machine.

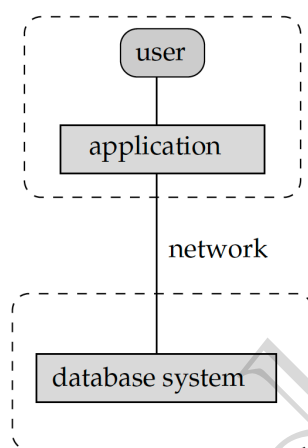


b. **T2 Architecture**

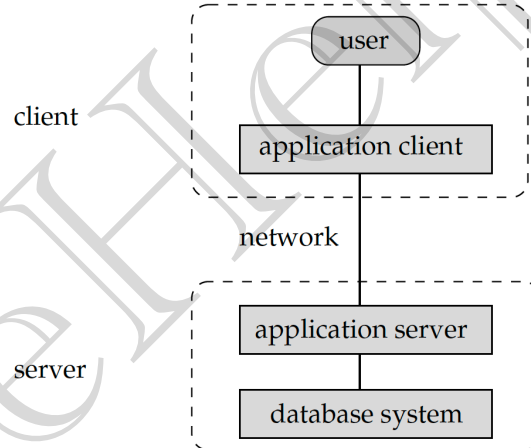
- i. App is partitioned into 2-components.
- ii. Client machine, which invokes DB system functionality at server end through query language statements.
- iii. API standards like **ODBC & JDBC** are used to interact between client and server.

c. **T3 Architecture**

- i. App is partitioned into 3 logical components.
- ii. Client machine is just a frontend and doesn't contain any direct DB calls.
- iii. Client machine communicates with App server, and App server communicated with DB system to access data.
- iv. **Business** logic, what action to take at that condition is in App server itself.
- v. T3 architecture are best for **WWW** Applications.
- vi. **Advantages:**
 - 1. **Scalability** due to distributed application servers.
 - 2. **Data integrity**, App server acts as a middle layer between client and DB, which minimize the chances of data corruption.
 - 3. **Security**, client can't directly access DB, hence it is more secure.

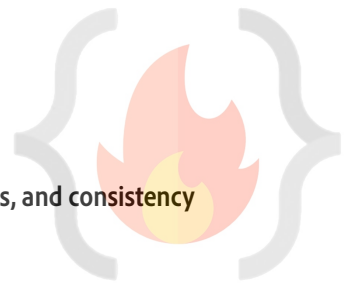


a. two-tier architecture



b. three-tier architecture

LEC-3: Entity-Relationship Model



1. **Data Model:** Collection of conceptual tools for **describing data, data relationships, data semantics, and consistency constraints**.
2. **ER Model**
 1. It is a high level data model based on a perception of a **real** world that consists of a collection of basic objects, called **entities** and of **relationships** among these objects.
 2. Graphical representation of ER Model is **ER diagram**, which acts as a **blueprint** of DB.
3. **Entity:** An Entity is a "**thing**" or "**object**" in the real world that is **distinguishable** from all other objects.
 1. It has **physical existence**.
 2. Each student in a college is an entity.
 3. Entity can be **uniquely** identified. (By a primary attribute, aka Primary Key)
 4. **Strong Entity:** Can be uniquely identified.
 5. **Weak Entity:** Can't be uniquely identified, depends on some other strong entity.
 1. It doesn't have sufficient attributes, to select a uniquely identifiable attribute.
 2. Loan -> Strong Entity, Payment -> Weak, as instalments are sequential number counter can be generated separate for each loan.
 3. **Weak entity depends on strong entity for existence.**
4. **Entity set**
 1. It is a set of entities of the **same** type that share the **same** properties, or attributes.
 2. E.g., Student is an entity set.
 3. E.g., Customer of a bank
5. **Attributes**
 1. An entity is represented by a set of attributes.
 2. Each entity has a value for each of its attributes.
 3. For each attribute, there is a set of **permitted values**, called the **domain**, or **value** set, of that attribute.
 4. E.g., Student Entity has following attributes
 - A. Student_ID
 - B. Name
 - C. Standard
 - D. Course
 - E. Batch
 - F. Contact number
 - G. Address
5. **Types of Attributes**
 1. **Simple**
 1. Attributes which can't be divided further.
 2. E.g., Customer's account number in a bank, Student's Roll number etc.
 2. **Composite**
 1. Can be divided into subparts (that is, other attributes).
 2. E.g., Name of a person, can be divided into first-name, middle-name, last-name.
 3. If user wants to refer to an entire attribute or to only a component of the attribute.
 4. Address can also be divided, street, city, state, PIN code.
 3. **Single-valued**
 1. Only one value attribute.
 2. e.g., Student ID, loan-number for a loan.
 4. **Multi-valued**
 1. Attribute having more than one value.
 2. e.g., phone-number, nominee-name on some insurance, dependent-name etc.
 3. Limit constraint may be applied, upper or lower limits.
 5. **Derived**
 1. Value of this type of attribute can be derived from the value of other related attributes.



2. e.g., Age, loan-age, membership-period etc.

6. NULL Value

1. An attribute takes a null value when an entity does not have a value for it.
2. It may indicate "not applicable", value doesn't exist. e.g., person having no middle-name
3. It may indicate "unknown".
 1. Unknown can indicate missing entry, e.g., name value of a customer is NULL, means it is missing as name must have some value.
 2. Not known, salary attribute value of an employee is null, means it is not known yet.

6. Relationships

1. **Association** among two or more entities.
2. e.g., Person has vehicle, Parent has Child, Customer borrow loan etc.
3. **Strong Relationship**, between two independent entities.
4. **Weak Relationship**, between weak entity and its owner/strong entity.
 1. e.g., Loan <instalment-payments> Payment.
5. **Degree of Relationship**
 1. Number of entities participating in a relationship.
 2. **Unary**, Only one entity participates. e.g., Employee manages employee.
 3. **Binary**, two entities participates. e.g., Student takes Course.
 4. **Ternary** relationship, three entities participates. E.g., Employee works-on branch, employee works-on job.
 5. Binary are **common**.

7. Relationships Constraints

1. Mapping Cardinality / Cardinality Ratio

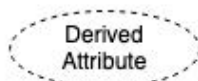
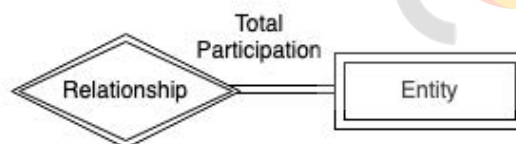
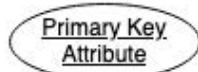
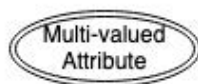
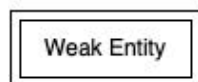
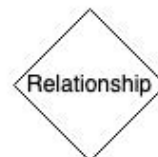
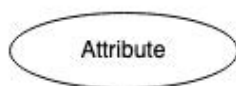
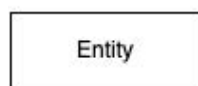
1. Number of entities to which another entity can be associated via a relationship.
2. **One to one**, Entity in A associates with at most one entity in B, where A & B are entity sets. And an entity of B is associated with at most one entity of A.
 1. E.g., Citizen has Aadhar Card.
3. **One to many**, Entity in A associated with N entity in B. While entity in B is associated with at most one entity in A.
 1. e.g., Citizen has Vehicle.
4. **Many to one**, Entity in A associated with at most one entity in B. While entity in B can be associated with N entity in A.
 1. e.g., Course taken by Professor.
5. **Many to many**, Entity in A associated with N entity in B. While entity in B also associated with N entity in A.
 1. Customer buys product.
 2. Student attend course.

2. Participation Constraints

1. Aka, **Minimum cardinality constraint**.
2. **Types**, Partial & Total Participation.
3. **Partial Participation**, not all entities are involved in the relationship instance.
4. **Total Participation**, each entity must be involved in at least one relationship instance.
5. e.g., Customer borrow loan, loan has total participation as it can't exist without customer entity. And customer has partial participation.
6. **Weak entity has total participation constraint, but strong may not have total.**

8. ER Notations

Symbols used in ER Diagram

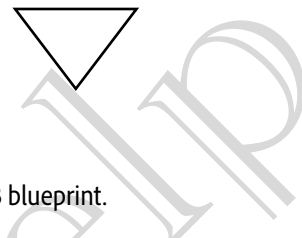


CodeHelp

LEC-4: Extended ER Features



1. **Basic ER Features** studied in the LEC-3, can be used to model most DB features but when complexity increases, it is better to use some Extended ER features to model the DB Schema.
2. **Specialisation**
 1. In ER model, we may require to subgroup an entity set into other entity sets that are distinct in some way with other entity sets.
 2. **Specialisation** is **splitting** up the entity set into further **sub entity sets** on the basis of their **functionalities, specialities and features**.
 3. It is a **Top-Down** approach.
 4. e.g., **Person** entity set can be divided into **customer, student, employee**. Person is **superclass** and other specialised entity sets are **subclasses**.
 1. We have **"is-a"** relationship between superclass and subclass.
 2. Depicted by **triangle** component.
 5. **Why Specialisation?**
 1. Certain attributes may only be applicable to a few entities of the parent entity set.
 2. DB designer can show the distinctive features of the sub entities.
 3. To group such entities we apply Specialisation, to overall **refine** the DB blueprint.
3. **Generalisation**
 1. It is just a **reverse** of Specialisation.
 2. DB Designer, may encounter certain properties of two entities are overlapping. Designer may consider to make a new generalised entity set. That generalised entity set will be a super class.
 3. **"is-a"** relationship is present between subclass and super class.
 4. e.g., **Car, Jeep and Bus** all have some common attributes, to avoid data repetition for the common attributes. DB designer may consider to Generalise to a new entity set **"Vehicle"**.
 5. It is a **Bottom-up** approach.
 6. **Why Generalisation?**
 1. Makes DB more **refined** and **simpler**.
 2. Common attributes are not **repeated**.
4. **Attribute Inheritance**
 1. **Both** Specialisation and Generalisation, has attribute inheritance.
 2. The attributes of higher level entity sets are inherited by lower level entity sets.
 3. E.g., **Customer & Employee** inherit the attributes of **Person**.
5. **Participation Inheritance**
 1. If a parent entity set participates in a relationship then its child entity sets will also participate in that relationship.
6. **Aggregation**
 1. **How to show relationships among relationships?** - Aggregation is the technique.
 2. **Abstraction** is applied to treat relationships as higher-level entities. We can call it Abstract entity.
 3. **Avoid redundancy** by aggregating relationship as an entity set itself.



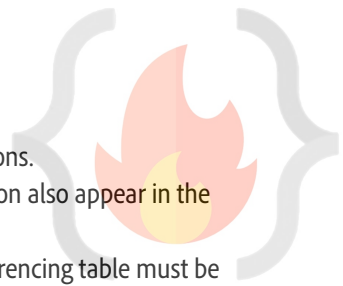
LEC-7: Relational Model



1. Relational Model (RM) organises the data in the form of **relations (tables)**.
2. A relational DB consists of **collection of tables**, each of which is assigned a **unique name**.
3. A **row** in a table represents a relationship among a set of values, and table is collection of such relationships.
4. **Tuple**: A single row of the table representing a single data point / a unique record.
5. **Columns**: represents the attributes of the relation. Each attribute, there is a permitted value, called **domain of the attribute**.
6. **Relation Schema**: defines the design and structure of the relation, contains the name of the relation and all the columns/attributes.
7. Common RM based DBMS systems, aka RDBMS: Oracle, IBM, **MySQL**, MS Access.
8. **Degree of table**: number of attributes/columns in a given table/relation.
9. **Cardinality**: Total no. of tuples in a given relation.
10. **Relational Key**: Set of attributes which can uniquely identify an each tuple.
11. Important **properties** of a **Table** in Relational Model
 1. The name of relation is distinct among all other relation.
 2. The values have to be atomic. Can't be broken down further.
 3. The name of each attribute/column must be unique.
 4. Each tuple must be unique in a table.
 5. The sequence of row and column has no significance.
 6. Tables must follow integrity constraints - it helps to maintain data consistency across the tables.
12. **Relational Model Keys**
 1. Super Key (**SK**): Any P&C of attributes present in a table which can uniquely identify each tuple.
 2. Candidate Key (**CK**): minimum subset of super keys, which can uniquely identify each tuple. It contains no redundant attribute.
 1. **CK value shouldn't be NULL.**
 3. Primary Key (**PK**):
 1. Selected out of CK set, has the least no. of attributes.
 4. Alternate Key (**AK**)
 1. All CK except PK.
 5. Foreign Key (**FK**)
 1. It creates relation between two tables.
 2. A relation, say r1, may include among its attributes the PK of an other relation, say r2. This attribute is called FK from r1 referencing r2.
 3. The relation r1 is aka **Referencing (Child) relation** of the FK dependency, and r2 is called **Referenced (Parent) relation** of the FK.
 4. FK helps to cross reference between two different relations.
 6. **Composite Key**: PK formed using at least 2 attributes.
 7. **Compound Key**: PK which is formed using 2 FK.
 8. **Surrogate Key**:
 1. Synthetic PK.
 2. Generated automatically by DB, usually an integer value.
 3. May be used as PK.
13. **Integrity Constraints**
 1. CRUD Operations must be done with some integrity policy so that DB is always consistent.
 2. Introduced so that we do not accidentally corrupt the DB.
 3. **Domain Constraints**
 1. Restricts the value in the attribute of relation, specifies the Domain.
 2. Restrict the Data types of every attribute.
 3. E.g., We want to specify that the enrolment should happen for candidate birth year < 2002.
 4. **Entity Constraints**
 1. Every relation should have PK. PK != NULL.

5. Referential Constraints

1. Specified between two relations & helps maintain consistency among tuples of two relations.
 2. It requires that the value appearing in specified attributes of any tuple in referencing relation also appear in the specified attributes of at least one tuple in the referenced relation.
 3. If FK in referencing table refers to PK of referenced table then every value of the FK in referencing table must be NULL or available in referenced table.
 4. FK must have the matching PK for its each value in the parent table or it must be NULL.
6. **Key Constraints:** The six types of key constraints present in the Database management system are:-
1. NOT NULL: This constraint will restrict the user from not having a NULL value. It ensures that every element in the database has a value.
 2. UNIQUE: It helps us to ensure that all the values consisting in a column are different from each other.
 3. DEFAULT: it is used to set the default value to the column. The default value is added to the columns if no value is specified for them.
 4. CHECK: It is one of the integrity constraints in DBMS. It keeps the check that integrity of data is maintained before and after the completion of the CRUD.
 5. PRIMARY KEY: This is an attribute or set of attributes that can uniquely identify each entity in the entity set. The primary key must contain unique as well as not null values.
 6. FOREIGN KEY: Whenever there is some relationship between two entities, there must be some common attribute between them. This common attribute must be the primary key of an entity set and will become the foreign key of another entity set. This key will prevent every action which can result in loss of connection between tables.



CodeHelp

LEC-8: Transform - ER Model to Relational Model



1. Both **ER-Model** and **Relational Model** are abstract logical representation of real world enterprises. Because the two models implies the similar design principles, **we can convert ER design into Relational design**.
2. Converting a DB representation from an ER diagram to a table format is the way we arrive at Relational DB-design from an ER diagram.
3. ER diagram **notations to relations**:
 1. **Strong Entity**
 1. Becomes an **individual table** with entity name, attributes becomes columns of the relation.
 2. Entity's Primary Key (PK) is used as Relation's PK.
 3. **FK** are added to establish relationships with other relations.
 2. **Weak Entity**
 1. A table is formed with all the attributes of the entity.
 2. PK of its corresponding Strong Entity will be added as **FK**.
 3. **PK** of the relation will be a composite PK, {FK + Partial discriminator Key}.
 3. **Single Values Attributes**
 1. Represented as **columns** directly in the tables/relations.
 4. **Composite Attributes**
 1. Handled by **creating a separate attribute** itself in the original relation for each composite attribute.
 2. e.g., **Address**: {street-name, house-no}, is a composite attribute in customer relation, we add address-street-name & address-house-name as new columns in the attribute and ignore "address" as an attribute.
 5. **Multivalued Attributes**
 1. **New tables** (named as original attribute name) are created for each multivalued attribute.
 2. PK of the entity is used as column **FK** in the new table.
 3. Multivalued attribute's similar name is added as a column to define multiple values.
 4. **PK** of the new table would be {FK + multivalued name}.
 5. e.g., For Strong entity **Employee**, **dependent-name** is a multivalued attribute.
 1. New table named dependent-name will be formed with columns emp-id, and dname.
 2. PK: {emp-id, name}
 3. FK: {emp-id}
 6. **Derived Attributes**: Not considered in the tables.
 7. **Generalisation**
 1. **Method-1**: Create a table for the higher level entity set. For each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the primary key of the higher-level entity set.
For e.g., Banking System generalisation of Account - saving & current.
 1. Table 1: account (account-number, balance)
 2. Table 2: savings-account (account-number, interest-rate, daily-withdrawal-limit)
 3. Table 3: current-account (account-number, overdraft-amount, per-transaction-charges)
 2. **Method-2**: An alternative representation is possible, if the generalisation is disjoint and complete—that is, if no entity is a member of two lower-level entity sets directly below a higher-level entity set, and if every entity in the higher level entity set is also a member of one of the lower-level entity sets. Here, do not create a table for the higher-level entity set. Instead, for each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the higher-level entity sets.
Tables would be:
 1. Table 1: savings-account (account-number, balance, interest-rate, daily-withdrawal-limit)
 2. Table 2: current-account (account-number, balance, overdraft-amount, per-transaction-charges)
 3. **Drawbacks of Method-2**: If the second method were used for an overlapping generalisation, some values such as balance would be stored twice unnecessarily. Similarly, if the generalisation were not complete—that is, if some accounts were neither savings nor current accounts—then such accounts could not be represented with the second method.
 8. **Aggregation**

1. Table of the relationship set is made.
2. Attributes includes primary keys of entity set and aggregation set's entities.
3. Also, add descriptive attribute if any on the relationship.



CodeHelp



Lec-8

* Transformation from ER model to Relational model.

Loan $\rightarrow$

Loan-number	amount
-------------	--------

weak Entity Payment $\Rightarrow$

P.K.

P.K.

Loan-number	payment no.	payment-date	payment-amount
-------------	-------------	--------------	----------------

* Composite attribute $\rightarrow \rightarrow$ sep. attribute for each component.

Customer table

customer-name	address-city	address-state	address-pincode	address-streetno.
				address-streetname

* Multi-value attribute

F.K



emp-id	dname.
--------	--------

dependent-name - new relation.

{ emp-id, dname }



P.K ..

* Generalization

Method 1

* Aggregation →

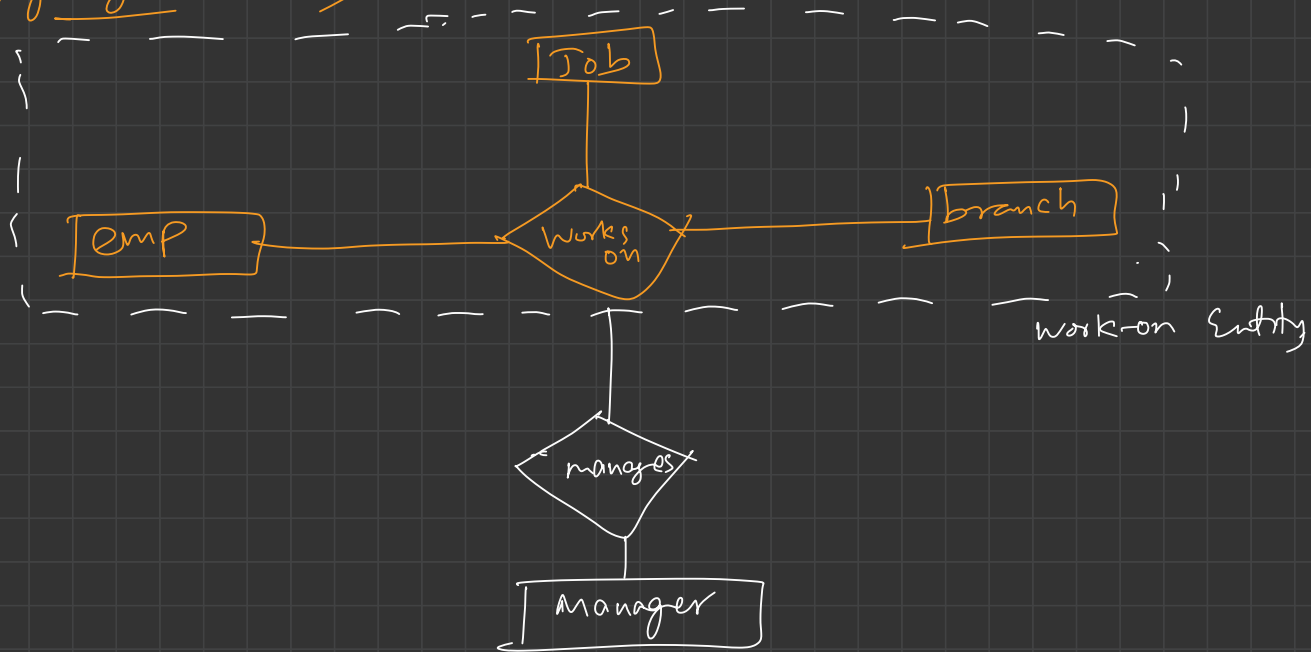
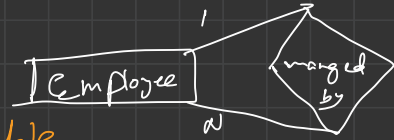


Table: manages (mgr-id, emp-id, job-id, branch-id) P.K.

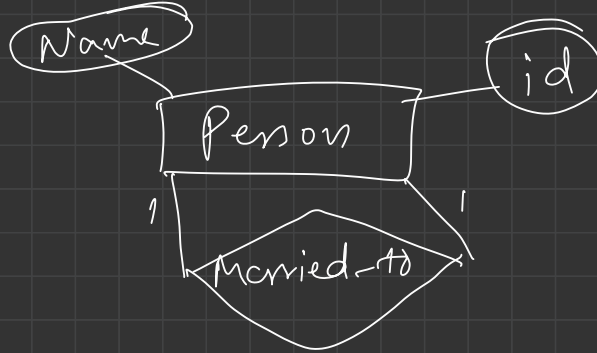
* Unary Relationship :-

we will add another attribute in Employee table,
which will be F.K.



Emp-id	name	joining date	Emp_mgr_id
201	---	---	205
202	---	---	205
205	---	---	Null.

* 1:1 →



Person(id, Name, spouse_id)
{ F.K }

* M:N $\rightarrow$

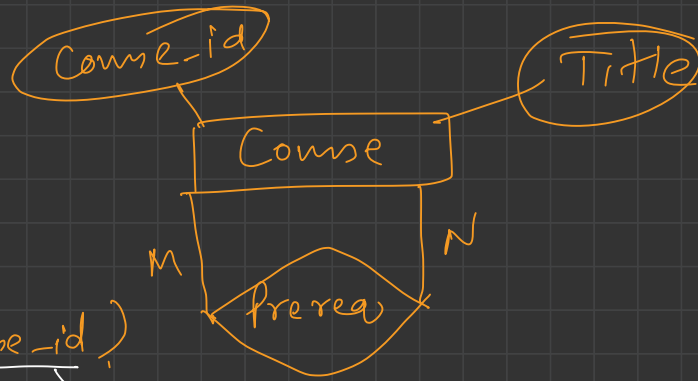
① Course (id $\rightarrow$ Title)

② prereq (id, prereq, -course-id)

$\downarrow$
{FK}

$\downarrow$
{FK}

$\rightarrow$ P.K



* FB Relational Model

- (1) user_profile (username, name-first, name-last, password, DOB)
- (2) user-profile-email (username {F.K}, email)
- (3) user-profile-contact (username {f.k}, contact-number)
- (4) friendship (profile_req {f.k}, profile_accept {f.k}) $\longrightarrow$ Compound key
- (5) post-like (post-like-id, timestamp, post-id {f.k}, username {f.k})
- (6) user-post (post-id, created-timestamp, modified-timestamp, text-content, username {f.k})
- (7) user-post-image (post-id {f.k}, image-url)
- (8) user-post-video (post-id {f.k}, video-url)
- (9) post-comment (post-comment-id, text-content, timestamp, post-id {f.k}, username {f.k})



Lec-9

* SQL in One video

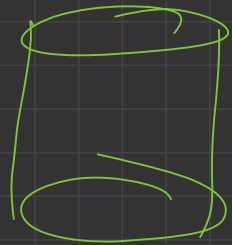
SQL $\Rightarrow$ Structured Query language

RDBMS $\Rightarrow$ MySQL, Oracle, MS access etc.

CRUD

SQL queries.

API
RDBMS

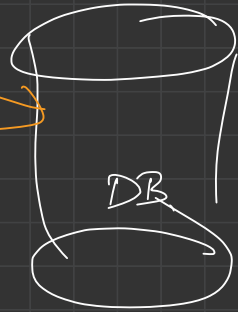


CRVD



Create
Read
Update
Delete.

20



RDBMS

RDBMS

RDBMS $\Rightarrow$ ① MySQL

② MSSQL server

⑤ Oracle

④ IBM.

→ Open some RDBMS.

→ Meta, Adobe etc.

SQL

- ① Query language.
- ② way to access data

MySQL

- ① MySQL itself a RDBMS
- ② CRUD done on it using SQL.

⇒ Install → ① MySQL community server
② MySQL workbench

2) workbench → DB create
Table

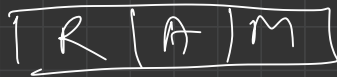
* SQL Data types

variable datatype

VARCHAR (255)

CHAR (255)

Arnd-
dohm



$2\sqrt{5}$

SIGNED, unsigned

TINYINT $\rightarrow$ (-128 to 127)

UNSIGNED TINYINT $\Rightarrow$ (0 $\rightarrow$ 255)

create TABLE table-name (

col1 INT,

col2 INT UNSIGNED

);

⇒ Advanced DT :-

① JSON → JS Object Notation

cr - {
; col1 JSON,
}

SQL Types of Commands

① DDL

⇒ DQL ⇒

- ① DB
- ② table

user defined tables

Student

Dual table

↳ without using FROM

SELECT

wild cards

⇒

' % pa % '

→ 'abcpadb'

'pabc'

'adpa'

'_pa_'

apab

bpa c

Sorting

select * from worker ORDER BY salary;

ASC (Default)

DESC (↓)

* Distinct values

— Department (distinct)

HR
HR
Admin
Admin,
Account



HR
Admin
Accounts

* Data Grouping

→ find no. of employees working in different depart

→ HR → ?

Account → ?

Admin → ?

→ Grouping → Aggregation
(count)

⇒ GROUP BY → Aggregation fⁿ.
COUNT, SUM
AVG, MIN, MAX.

Worker

Name	Department
Monika	HR
Nam.	Admin
C	
D	HR
E	Admin
F	Admin
G	Account
H	Account
	Admin

Name	Department
Monika	HR
C	HR
"	Admin
"	Admin
"	Admin
"	Account
"	Account

Q find avg. salary per department?

* HAVING

→ select where to filter

"GROUP BY" HAVING

2) department, count, having More than 2 workers

* WHERE VS HAVING

* DDL Constants

① Primary Key Constraint

- Not Null
- Unique
- Only one P.K

Good practice

→ P.K INT

② Foreign Key

→ Fk refers P.K of other table.

work bench

③ Unique

```
create table Customer (  
    ;  
    Name VARCHAR(255) UNIQUE,  
    ;  
);
```

④ CHECK

```
create table account (  
    ;  
    CONSTRAINT acc_balance_chk CHECK (  
        balance > 0.00)  
    ;  
);
```

④ DEFAULT

create table account (

!

balance INT NOT NULL Default 0,

!.

)

* DML Data Modification Language.

* Referential Constraint

① INSERT

② Delete Constraint

→ ON Delete Cascade
→ ON Delete Set Null.

Replace :-

- ① Data already present, Replace.
- ② Data not present, INSERT

Replace Vs Update

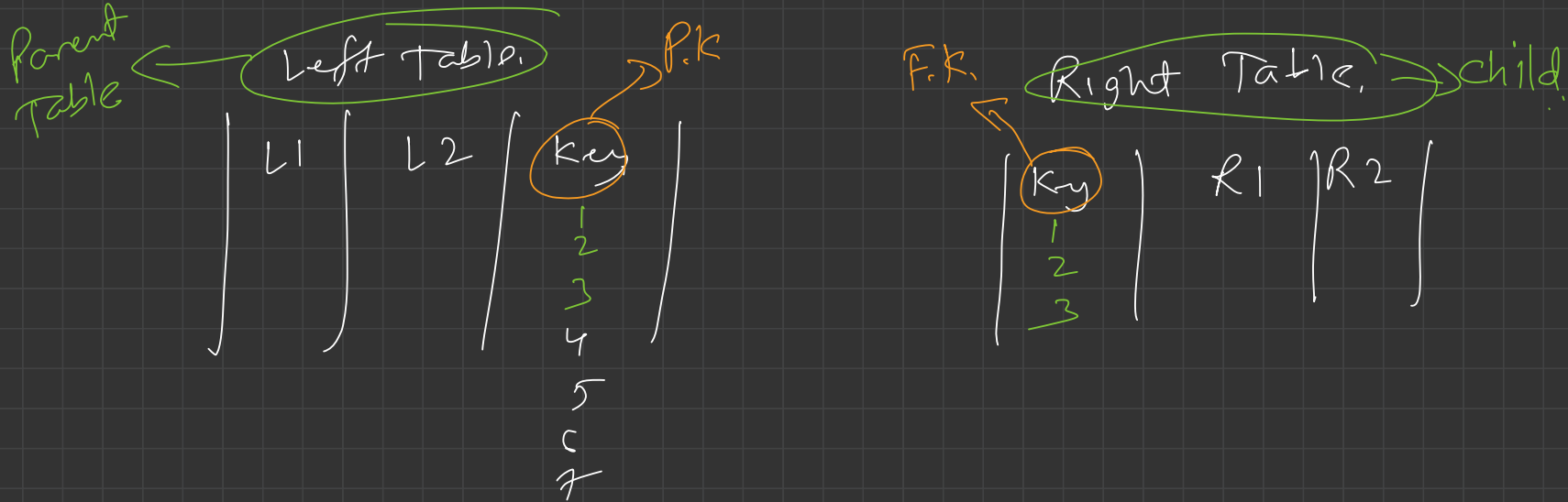
→ if row is not present, Replace will add a new row while UPDATE will do Nothing

#

JOINS

⇒ F.K → Relations

→ we → Data fetch → JOINS



① INNER JOIN

Resultant table

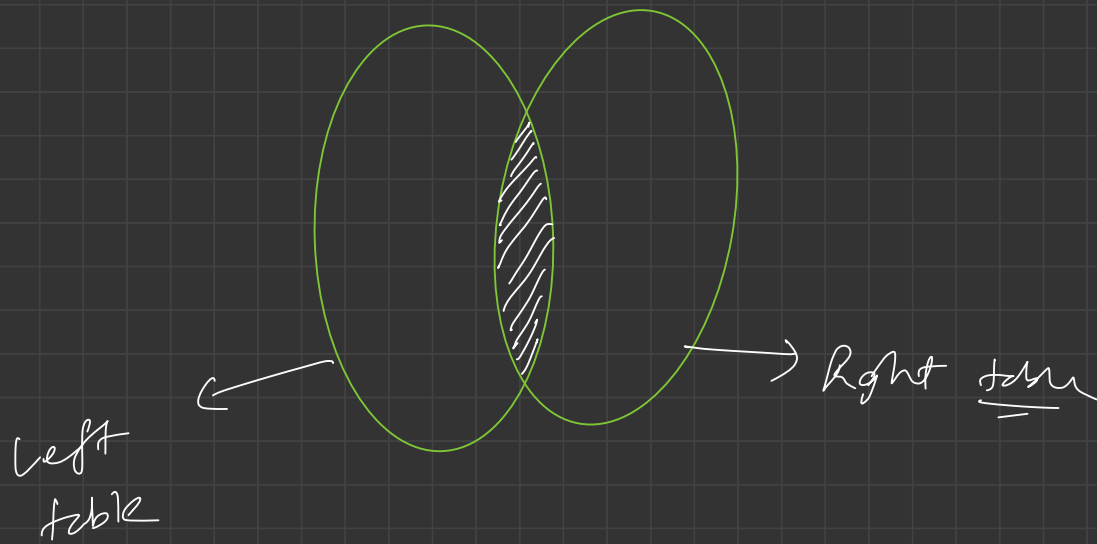
key	L1	L2	R1	R2
1				
2				
3				

— Join → To apply joins, there should be a common attribute.

Syntax!

Select C.*, O.* FROM Customer AS C INNER JOIN Orders AS O
ON c.id = o.cust-id;

Annotations:
- C.* points to "C.id, C.name."
- O.* points to "O.product."
- AS points to "aliasing".



INNER JOIN

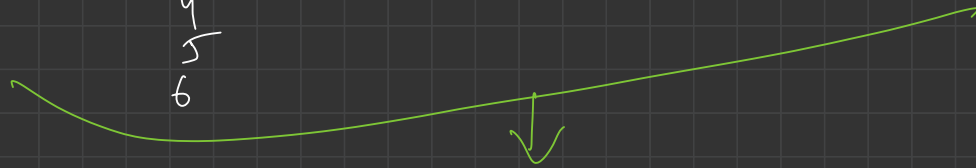
⑦ Left Join

Left Table

L1	L2	key
		1
		2
		3
		4
		5
		6

R Table

key	R1	R2
1		
2		
3		



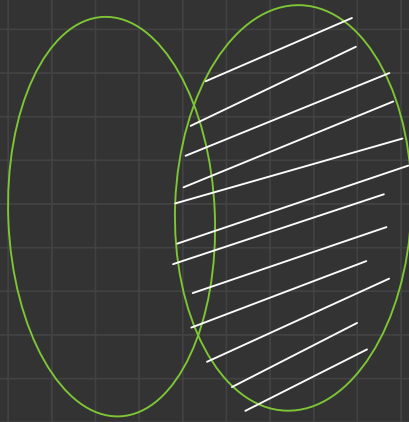
Resultant

key	L1	L2	R1	R2
1				
2				
3				
4			Null	Null
5			Null	Null
6				

select c.*, o.* FROM Customer as C LEFT JOIN
Orders as O
ON c.id = o.cust-id;

③ Right join

— intended in knowing only the
right side info

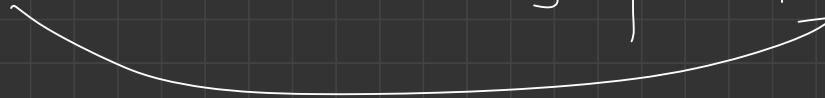


Left table

L1	L2	Key
		1
		2
		3

Right table

Key	R1	R2
1		
2		
3		
4		
5		



Result

Key	L1	L2	R1	R2
1				
2				
3				
4	Null	Null		
5	Null	Null		

④ Full join

Left

L1	L2	key
		1
		2
		3
		4
		5
		6

Right

key	R1	R2
1		
2		
3		
7		
8		
9		

MySQL Full JOIN X keyword X.

Simulated

Left Join $\cup$ Right Join

Syntax

select * from lefttable as l LEFT JOIN righttable as r
ON l.key = r.key;

UNION

select * from lefttable as l RIGHT JOIN righttable as r
ON l.key = r.key;

⊗ Cross join

→ Cartesian product

→

T_L
5 rows

T_R
10 rows

Result 1 - $5 \times 10 = 50$ rows

eg.

L_1	L_2
1	A
2	B

R_1	R_2
3	C
4	D

2) Result
=

→ Industrial use X.

L_1	L_2	R_1	R_2
1	A	3	C
1	A	4	D
2	B	3	C
2	B	4	D

⑤ Self Join

- emulate.
- 'INNER JOIN'
- Alias 'AS'

eg. select e1.id, e2.id, e2.name
FROM employee as e1
INNER JOIN
employee as e2
ON e1.id = e2.id;



→ Work bench senior
* Project

* Project

	id	empID	name	startdate	clientID
▶	1	1	A	2021-04-21	3
	2	2	B	2021-03-12	1
	3	3	C	2021-01-16	5
	4	3	D	2021-04-27	2
	5	5	E	2021-05-01	4
	NULL	NULL	NULL	NULL	NULL

* EMPLOYEES

[illegible]

~~A~~ CLIENT

[illegible]

Q can we ~~we~~ join w/o using JOIN keyword?

→ Yes

Syntax

```
select * FROM lefttable, Righttable WHERE  
lefttable.id  
= Righttable.id;
```

* Set Operations

{ 1, 2, 3, 4, 5, 6, 7, 8 }

Table 1

col 1	col 2
A	1
B	1
C	2

Table 2

col 1	col 2
A	1
B	2
D	3

① Union

② Intersection

③ MINUS

→ UNION
=

$T_1 \cup T_2$

$T_1 \cup T_2 \rightarrow$

col 1	col 2
A	1
B	1
C	2
B	2
D	3

— ROWS.

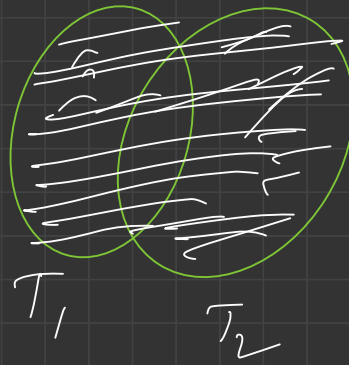
Syntax! →

Select * FROM Table 1

UNION

Select * FROM Table 2;

Set operation
→ Combine
two or more
select statements



JOIN vs UNION

Full Join :-

Col 1	Col 2	Col 1	Col 2
A	1	A	1
B	1	B	2
C	2	Null	Null
Null	Null	D	3

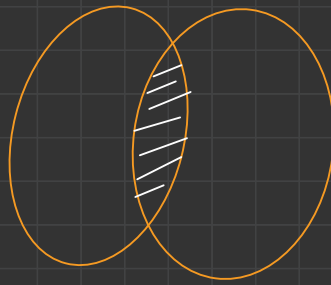
② INTERSECT

select * from T₁

INTERSECT

X

select * from T₂;



→ Emulate

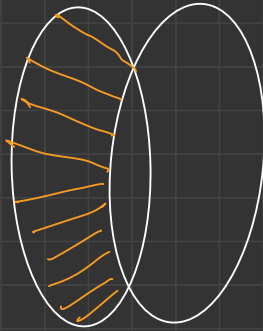
select DISTINCT id from T₁

INNER JOIN T₂

using (id);

③ MINUS

$T_1 - T_2$



Syntax:-> `SELECT id FROM T1
LEFT JOIN T2 USING(id)
WHERE T2.id IS NULL;`

Example

ABC

Dept 1

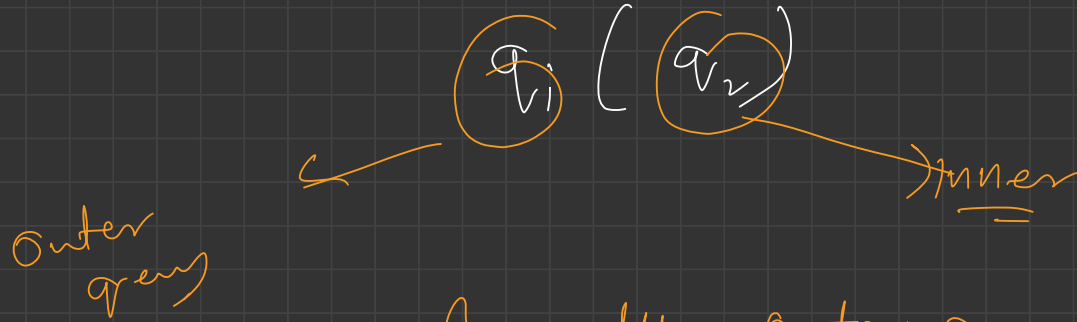
	empid	name	role
▶	1	A	engineer
	2	B	salesman
	3	C	manager
	4	D	salesman
	5	E	engineer
	NULL	NULL	NULL

Dept 2.

	empid	name	role
▶	3	C	manager
	6	F	marketing
	7	G	salesman
	NULL	NULL	NULL

Sub Queries

— alternative to joins



Generally, outer query depends on inner query

eg. `select * from table where id IN (select id
from table
where name = 'lat');`

→ Work bench senior
* Project

	id	empID	name	startdate	clientID
▶	1	1	A	2021-04-21	3
	2	2	B	2021-03-12	1
	3	3	C	2021-01-16	5
	4	3	D	2021-04-27	2
	5	5	E	2021-05-01	4
	NULL	NULL	NULL	NULL	NULL

* EMPLOYEES

[illegible]

CLIENT

[illegible]

* Correlated Subquery

Outer (inner),

— inner query that refers the outer query =

```

-- Correlated subquery
-- find 3rd oldest employee
SELECT *
FROM Employee e1
WHERE 3 = (
    SELECT COUNT(e2.age)
    FROM Employee e2
    WHERE e2.age >= e1.age
);

```

for each, e1.age inner → completely one time

① e1.age = 32

inner

⇒ 2

② e1.age = 44

⇒ ∅

③ 22 ⇒ 4

32
44
22
31
21

① e1.age = 31 ⇒ ③

SQL Views

⇒ MySQL → views

Customer ⇒ id | name | age | address |

view → name | age → alias — custom view

2) SQL

2) ~~AF~~ 50 Question Interview specific
ON SQL.

Home work

1
2

Note making
Practice